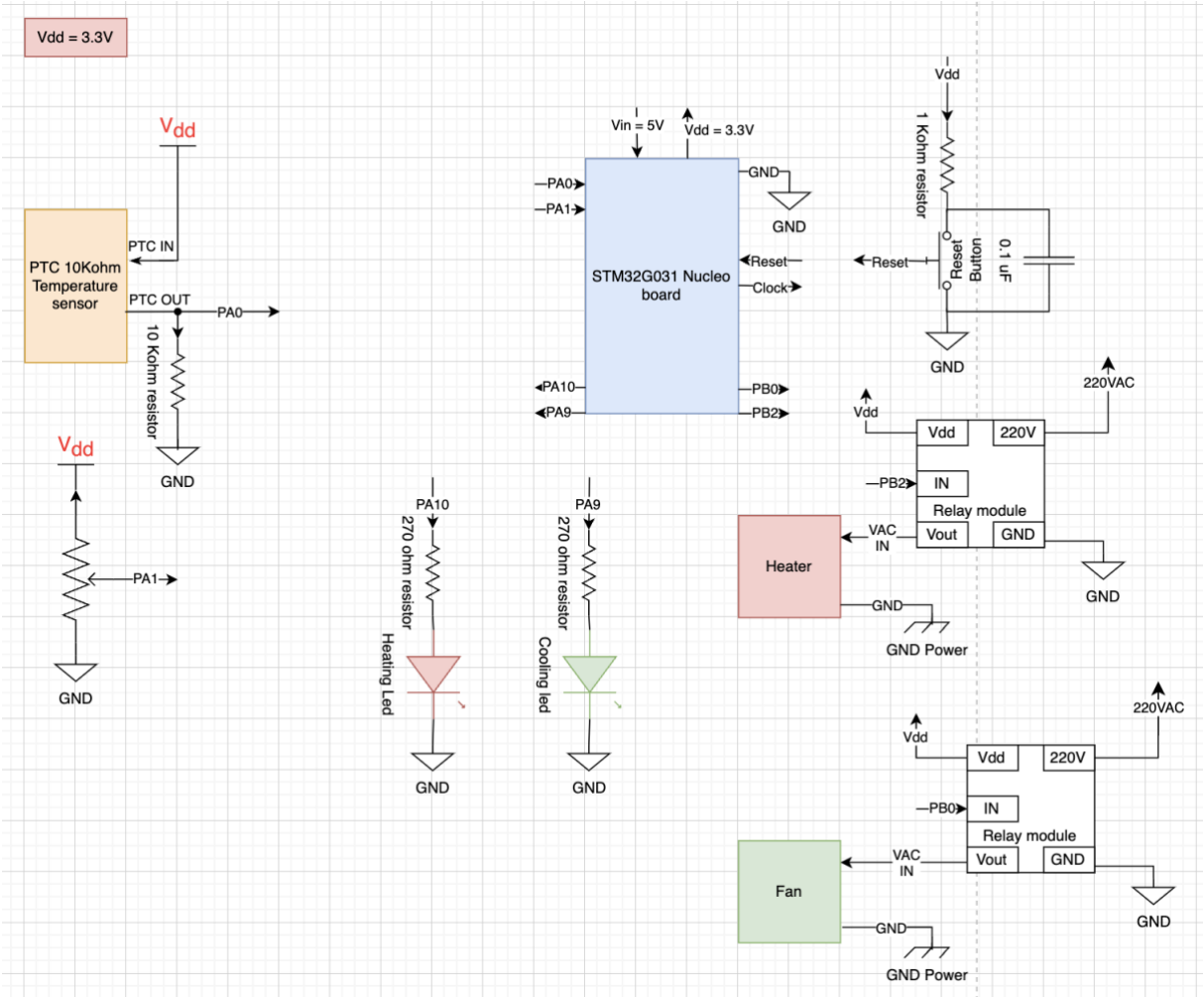
Fouad Aladhami
210102002808

Microprocessors HM 1

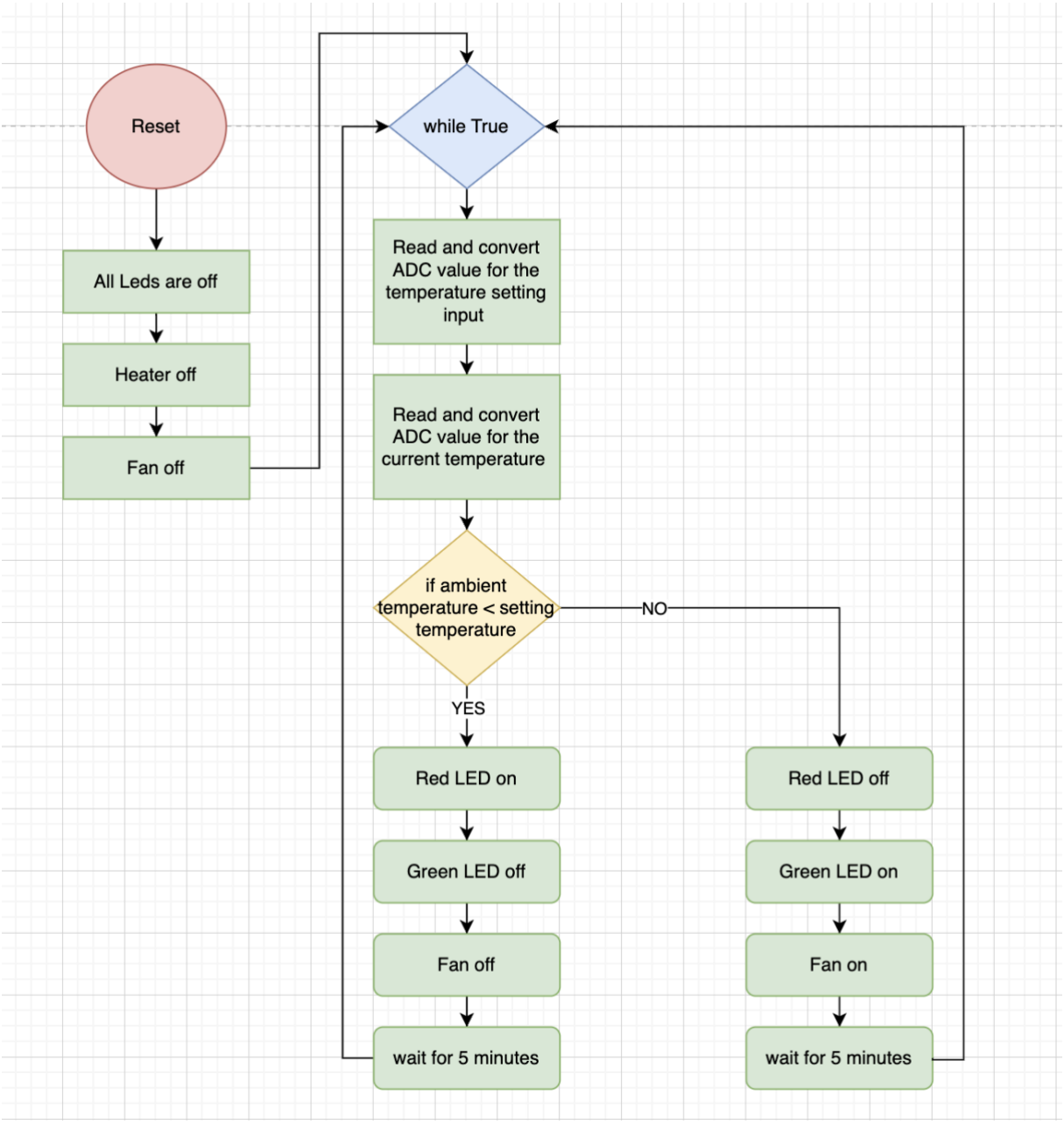
1.a) In this question the drop voltage of the led is assumed as 2 V. From V = I*R formula ,

(3.3-2)/(5mA) = R, so R will be 260 ohm, so I choose 270 ohm resistor because 260 ohm resistor is not sold (not produced) on the market.



Fouad Aladhami
210102002808

b)



Fouad Aladhami
210102002808

2.a) **MOVS R1, R0, LSL #4**

LSL is Logical Shift Left operator. And MOVS is the mnemonic for the instruction, which stands for "Move with S." The "S" indicates that this instruction updates the condition flags (N and Z).

After when the execution of this code line the value of R1 will be 0x23456780 because it shifted to the left and the register is 32-bit register, the Z flag is 0 because the result is not zero and the N flag is 0 because the most significant bit of result is not 1, it's 0, It's not indicate a negative number.

b) **MOVS R2,R1,LSR#8**

LSR is the logical shift right (LSR) operation, where the value in R1 will be shifted right by 8 bits. The value in R2 will be 0x 00234567. The Z flag will be 0 because the value is not zero, and the N flag will be 0 because the most significant bit is not 1. It's not indicate a negative number.

c) **MOVS R3, R0, ASR #8**

ASR is an arithmetic shift right (ASR) operation, where the value in R0 will be shifted right by 8 bits. The value in R3 will be 0x00123456. Because the arithmetic shift right if the most significant bit is 1 it will fill the shifted area with ones but here the most significant bit is 0. The Zero flag (Z) may be cleared if the result is non-zero and the Negative flag (N) will be 0 because the most significant bit of the result is 0, not indicating a negative number.

d) **MOVS R4, R0, ROR #4**

ROR is a rotate right (ROR) operation, where the value in R0 will be rotated right by 4 bits. Rotating right by 4 bits is equivalent to moving the 4 least significant bits to the most significant positions. So, after ROR #4, the result is 0x81234567. The Zero flag (Z) will be 0 because the result is non-zero, and the Negative flag(N) will be set because the most significant bit of the result is 1, indicating a negative number.(The number is equal to 10000001001000110100010101100111 in binary).

e) **MOVS R5, R0, LSL #29**

This code when execute logical shift left R0 and assign to R5 register. 0x12345678 is equal to 00010010001101000101011001111000 in binary, so when this value shifted to the left 29 bits, R5 will be equal to 0x00000000 and the Z flag will be set because the value of the register is 0.

The test code is written as follow and it is debugged in "cpulator" assembly simulator.

```
.global _start
_start:
        ldr R0,=#0x12345678
        MOVS R1,R0,LSL#4
        MOVS R2,R1,LSR#8
        MOVS R3,R0,ASR#8
        MOVS R4,R0,ROR #4
        MOVS R5, R0, LSL #29
```

Fouad Aladhami
210102002808

3) In little endian memory system, let's assume an 32 bit integer, the less significant byte is write first to the memory. Example:
Assume the int is equal to 0x12345678

Adress	Contents
0x100	0x78
0x101	0x56
0x102	0x34
0x103	0x12

In the question the memory address and contents are as following:

Address	Contents
0x24	0x06
0x25	0xFC
0x26	0x03
0x27	0xFF

And R1 contains 0x24.

a) LDRSB R2, [R1]
This command read a byte of memory at the address stored in R1 and write it to R2 register. This mean that it's read the data stored in 0x24, it is 0x06 and write it to R2 so R2 will be equal to 0x06. R2 = 0x00000006.

b) LDRSH R2, [R1]
This command read a half word (two bytes) of memory at the address stored in R1 and write it to R2 register. So, this code will read the contents of addresses 0x24, and 0x25 and because the memory is little endian the content of 0x24 will be the less significant. The half word msb is equal to one so while writing to R2 sign extension will be added. So R2 will be equal to 0xFFFFFC06. R2 = 0xFFFFFC06.

c) LDR R2, [R1]
This command read a word (four bytes) of memory at the address stored in R1 and write it to R2 register. So, this code will read the contents of addresses 0x24, 0x25, 0x26, 0x27 and because the memory is little endian the content of 0x24 will be the less significant. So R2 will be equal to 0x0xFF03FC06. R2 = 0xFF03FC06.

4)

a)

```
ADDS R6,R1,#0xEA @Add to R1 0xEA and write it to R6

MOVS R0,R6 @Write the content of R6 to R0
```

b)

```
LDR R6,=0xDEADBEEF @load the value 0xDEADBEEF to R6

SUBS R3,R6,R2 @R3 = R6-R2
```

c)

```
LDR R6,=0xE1EC0334 @load the value 0xE1EC0334 to R6

ADD R4,R6,R3 @R3 = R6-R2
```

d)

```
LDR R6,=334 @load the value 0xE1EC0334 to R6

MVN R6,R6 @take the not of R6 and rewrite to R6

ADD R4,R6,#1 @add 1 to R6 and write to R4
```